Module 3

16/03/26

Syntax directed translation

CFG + Semantic rule → to compute the attributes
↓
attribute grammar

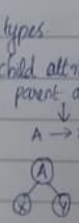
- Syntax directed definitions
- translation schema

Syntax directed translation (SDD) (definitions)

- attributes can be of 2 types

* Synthesized attribute → child attribute will be used to compute parent attribute

$A \to XY$



A.val = X.val + Y.val

* Inherited attribute

→ parent or/and sibling attribute will be used to compute the inherited attribute

$A \to XY$

Y.val = 2 * A.val

Synthesized translation

$E \to E^{(1)} + E^{(2)}$ $\quad E.val = E^{(1)}.val + E^{(2)}.val$

Inherited translation

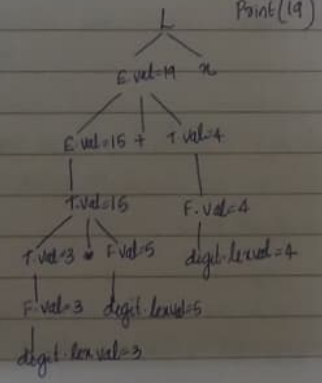
$A \to XYZ$, ~~Y.val~~ = 2 * A.VAL $\quad$ Y.VAL = 2 * A.VAL

SDD

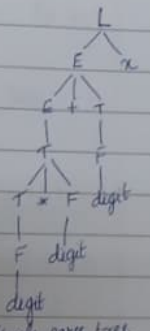
Production	Semantic rule
$L \to En$	print(E.val)
$E \to E+T$	E.val = E1.val + T.val
$E \to T$	E.val = T.val
$T \to T*F$	T.val = T1.val * F.val
$T \to F$	T.val = F.val
$F \to digit$	F.val = digit.lexval

eg: Draw parse tree & annotation

3 * 5 + 4n



Print(19)



Simple parse tree
↳ add semantic rule

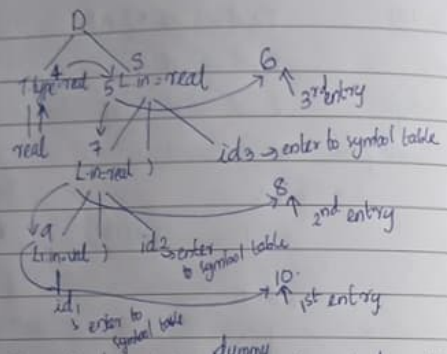
SDD with inherited attribute

Production	Semantic rule
$D \to TL$	L.in = T.type
$T \to int$	T.type = integer
$T \to real$	T.type = real

$L \to L, id$ addtype(id.entry, L.in)

$L \to id$ addtype(id.entry, L.in)

real id_1, id_2, id_3



- whether a fn is found, we hv a dummy synthesized attribute
- dependency graph ⟹ one node depends on another.

  c^* ⎫ means b depends on c
  b^* ⎭

- addtype(id.entry, L.in) in this case adds the symbol & type to symbol table.
- Above eg has both synthesized & inherited attribute

13/03/26

L-attributed definition

In this case both synthesized & inherited attribute can be there. But in inherited there is some restrictions.

1. $A \to X_1 X_2 \cdots X_n \quad 1 \le X_i \le n$

   $A \to X_1 X_2 \cdots X_{j-1}$

2. Inherited attribute of A

Eg

production	Semantic rule
$A \to LM$	$L.i = l(A.i)$
	$M.i = m(L.s)$
	$A.s = f(M.s)$
$A \to QR$	$R.i = r(A.i)$
	$Q.i = q(R.s)$
	$A.s = f(Q.s)$

2nd one is non L-attributed

Translation Schemes

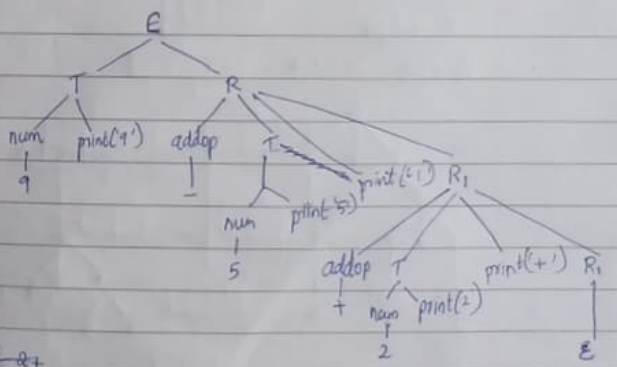
Along with the ~~Sem~~ CFG the semantic rule is written

$E \to TR$

$R \to addop\ T\ \{print(addop.lexeme)\}\ R_1 \mid \varepsilon$

$T \to num\ \{print(num.val)\}$

$9 - 5 + 2$



→ ~~Eg 2.3 Q5 Q7~~

~~following notation of given experiment~~

95-2+

↳ postfix notation of given expn

Intermediate code

Benefits

- given as input to different machine architecture
- A m/c independent code optimizer can be generated

3 types

1. Syntax tree 2. postfix notation 3. three-address code

Syntax tree

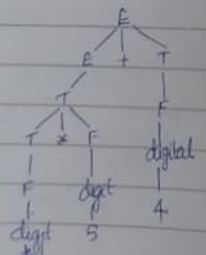
Abstract form of parse tree

$E \rightarrow E+T/T$

$T \rightarrow T*F/F$

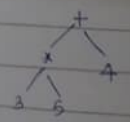
$F \rightarrow digit$

Ip string: 3*5+4



parse tree

Syntax tree



constructing syntax tree for expressions

3 functions

1. mknode (op, left, right)
2. mkleaf (id, entry)
3. mkleaf (num, val)

eg: a-4+c

P_1 = mkleaf (id, entry a)

P_2 = mkleaf (num, 4)

P_3 = mknode ('-', P_1, P_2)

P_4 = mkleaf (id, entry c)

P_5 = mknode ('+', P_3, P_4)

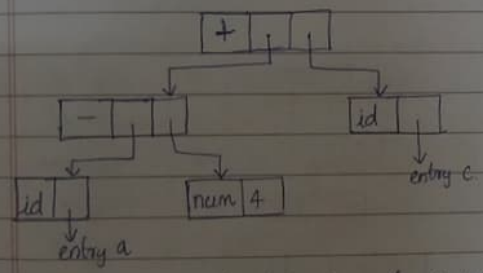


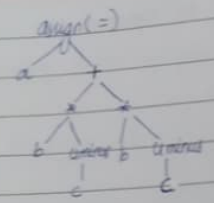
Fig Syntax tree for a-4+c

18/08/26 Directed acyclic graph (DAG) for expressions

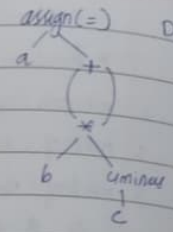
* DAG identifies the common subexpression.
* More than one ~~point~~ parent for common subexpression.

a = b*-c + b*-c

Syntax tree

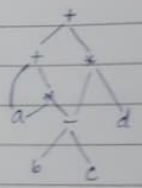


DAG

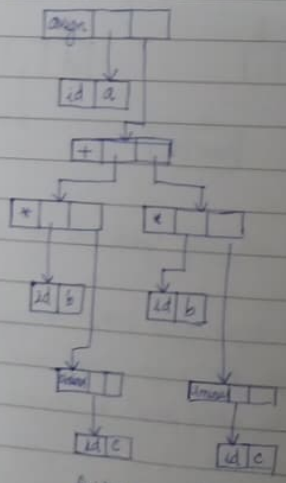


eg: $a+a*(b-c)+(b-c)*d$

DAG



eg: $a=b*-c+b*-c$



0	id	b	
1	id	c	
2	uminus	1	
3	*	0	2
4	id	b	
5	id	c	
6	uminus	5	
7	*	4	6
8	+	3	7
9	id	a	
10	assign	9	8

fig: bin representation of the syntax tree for $a=b*-c+b*-c$

Three address statements

eg: $x=y$ op z

$x=$ op y

$x=y$

if x relop y goto L

Implementation of 3 address statements

1. Quadruples – 4 fields will be used in table
2. Triples
3. Indirect triples – indirectly refering the triple table

eg: $a=b*-c+b*-c$

Quadruples

	op	arg1	arg2	Result	
(0)	uminus	c	–	t1	-c
(1)	*	b	t1	t2	b*-c
(2)	uminus	c	–	t3	
(3)	*	b	t3	t4	
(4)	+	t2	t4	t5	
(5)	=	t5		a	

Triples

	OP	arg1	arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

using index

Indirect triple

Statement			OP	arg1	arg2
(0)	(14)	(14)	uminus	c	
(1)	(15)	(15)	*	b	(14)
(2)	(16)	(16)	uminus	c	
(3)	(17)	(17)	*	b	(16)
(4)	(18)	(18)	+	(13)	(17)
(5)	(19)	(19)	assign	a	(18)

Comparison

Code optimization possible in quadruples, Indirect triples.

Space requirement - least in triples.

↳ Same in both quadruples & indirect triples.